

Operating Systems Lab

Week 5: Classic Synchronizing Problems

1. Producer-Consumer Problem

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 10

typedef struct Item
{
    int value;
} Item;

Item buffer[BUFFER_SIZE];
int in = 0, out = 0;
int counter = 0;

// producer section
void producer()
{
    while (counter == BUFFER_SIZE)
        ;

    Item nextProduced;
    printf("Enter value to produce: ");
    scanf("%d", &nextProduced.value);

    buffer[in] = nextProduced;
    printf("Produced %d at position %d.\n", nextProduced.value, in);

    in = (in + 1) % BUFFER_SIZE;
    counter++;
}

void consumer()
{
    while (counter == 0)
        ;
}
```

```

    Item nextConsumed = buffer[out];
    printf("Consumed %d from position %d.\n", nextConsumed.value, out);

    out = (out + 1) % BUFFER_SIZE;
    counter--;
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Produce\t 2. Consume\t 3. Exit.\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
            case 3:
                printf("Exiting.");
                break;

            default:
                printf("Error: Invalid choice.");
                exit(0);
                break;
        }
    }

    return 0;
}

```

Output:

```
1. Produce      2. Consume      3. Exit.
Enter your choice: 1
Enter value to produce: 10
Produced 10 at position 0.

1. Produce      2. Consume      3. Exit.
Enter your choice: 1
Enter value to produce: 20
Produced 20 at position 1.

1. Produce      2. Consume      3. Exit.
Enter your choice: 1
Enter value to produce: 30
Produced 30 at position 2.

1. Produce      2. Consume      3. Exit.
Enter your choice: 2
Consumed 10 from position 0.

1. Produce      2. Consume      3. Exit.
Enter your choice: 2
Consumed 20 from position 1.

1. Produce      2. Consume      3. Exit.
Enter your choice: 3
Exiting.
```

2. Producer-Consumer Problem using Semaphores

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;           // counts empty slots
sem_t full;             // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

void *producer(void *arg)
{
    int item, i = 0;

    while (1)
    {
        item = i++; // produce an item (here just incrementing number)

        sem_wait(&empty);           // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); // Leave critical section
        sem_post(&full);              // signal that new item is
        // available
        sleep(1);                     // simulate time to produce
    }
}

void *consumer(void *arg)
{
    int item;

    while (1)
    {
        sem_wait(&full);              // wait for filled slot
```

```

    pthread_mutex_lock(&mutex); // enter critical section

    item = buffer[out];
    printf("Consumed: %d from buffer[%d]\n", item, out);

    out = (out + 1) % BUFFER_SIZE;
    pthread_mutex_unlock(&mutex); // leave critical section
    sem_post(&empty);             // signal that a slot is free
    sleep(2);                     // simulate time to consume
}
}

int main()
{
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Output:

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
```

3. Dining-Philosophers Problems

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void *philosopher(void *num)
{
    int id = *(int *)num;
    int left = id;           // Left fork index
    int right = (id + 1) % N; // right fork index

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking

        // To avoid deadlock, philosophers with even id pick left then
        // with odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id,
left);

            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id,
right);
        }
        else
        {
            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id,
right);
        }
    }
}
```

```

        // Pick up left fork
        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id,
left);
    }

    // Eating
    printf("Philosopher %d is eating.\n", id);
    sleep(2); // Put down forks

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d.\n", id, left,
right);
}
}

int main()
{
    int i;
    int ids[N]; // Initialize forks (mutexes)

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++)
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);

    // Join threads (will never happen here, but good practice)
    for (i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    // Destroy mutexes (unreachable here)
    for (i = 0; i < N; i++)
        pthread_mutex_destroy(&forks[i]);

    return 0;
}

```

Output:

```
Philosopher 2 is thinking.  
Philosopher 1 is thinking.  
Philosopher 0 is thinking.  
Philosopher 4 is thinking.  
Philosopher 3 is thinking.  
Philosopher 3 picked up right fork 4.  
Philosopher 3 picked up left fork 3.  
Philosopher 3 is eating.  
Philosopher 1 picked up right fork 2.  
Philosopher 1 picked up left fork 1.  
Philosopher 1 is eating.  
Philosopher 0 picked up left fork 0.
```